Artificial Intelligence Internship

Skillify



Task 3 Report

Real World Data Analysis System

Submitted by

Nayab Maryam

Submitted to

Mam Momina Arshad

Task 3: Real World Dataset Analysis System:

Objective

Analyze a real-world dataset using and identify meaningful patterns and findings.

Requirements

- Python
- NumPy
- Pandas
- CSV dataset
- DataFrames
- Data filtering
- Data sorting
- Basic statistical analysis

Submission Format

- Jupyter Notebook / Google Colab (.ipynb)
- Analysis screenshots
- At least 7 meaningful findings

Solution:

First as dataset isnot given with the help of AI I downloaded a dataset using this link Superstore Dataset this is link of kaggle. I used this dataset as it is real world dataset.

This dataset contains information such as:

- Order ID and Order Date
- Customer Name and Segment
- Product Category and Sub-category
- Sales and Quantity
- Discount and Profit
- Region and State

Why this dataset?

We can investigate questions like:

1. Which product category generates the highest sales?
2. Which region generates the highest profit?
3. What is the average order value?
4. Which products have the highest sales?
5. How do discounts affect profitability?
6. Which customer segment contributes the most revenue?
7. Which products generate losses?

Then I created a file named Task_03_Sales_Data_Analysis.ipynb for this task then , I imported two libraries **NUMPY** and **PANDAS** .

Numpy: It stands for Numerical Python and used for numerical calculations .

Pandas: It's used for data manipulation and analysis like reading CSV ,creating dataframes, filtering and sorting records and calculating statistics.

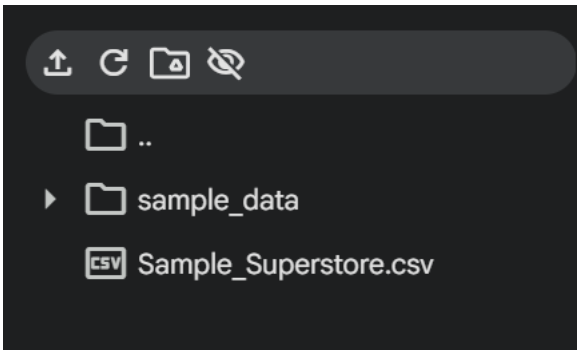
```
import numpy as np
import pandas as pd
```

The version I imported libraries are

```
print("NumPy Version:", np.__version__)
print("Pandas Version:", pd.__version__)

NumPy Version: 2.1.3
Pandas Version: 2.2.3
```

After it I upload my sales dataset csv file into folder :



Then load it using pd.read_csv() function and I used encoding 'cp1252' the reason is below:

Use encoding='cp1252' because our csv hasnot some characters of UTF-8 format.

By default, Pandas uses UTF-8 encoding when reading CSV files.

However, some CSV files are saved using different character encodings, such as Windows-1252 (also called cp1252).

Then I used df.head()) here df is the variable u can I used to read the csv.

This function shows me first 5 rows of my csv file data:

	ID	ID	Date	Ship Date	Mode	ID	Name	Segment	Country	City	...	Code	Region	ID	Category	Category	Name	Sales	Quanti
0	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-BO-10001798	Furniture	Bookcases	Bush Somerset Collection Bookcase	261.9600	
1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-CH-10000454	Furniture	Chairs	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400	
2	3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	...	90036	West	OFF-LA-10000240	Office Supplies	Labels	Self-Adhesive Address Labels for Typewriters b...	14.6200	
3	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311	South	FUR-TA-10000577	Furniture	Tables	Bretford CR4500 Series Slim Rectangular Table	957.5775	
4	5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311	South	OFF-ST-10000760	Office Supplies	Storage	Eldon Fold 'N Roll Cart System	22.3680	

5 rows × 21 columns

Then we checked the dimensions of dataset using Shape it tells no of rows and no of columns of dataset that are :

```
df.shape
(9994, 21)
```

Then we check the names of columns using df.columns and df.info to find the datatypes of each column which type of data int,string it is.

Check Names of Columns:

[15] 0s

```
df.columns
```

```
Index(['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode',
       'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State',
       'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category',
       'Product Name', 'Sales', 'Quantity', 'Discount', 'Profit'],
      dtype='object')
```

Check the DataTypes:

It tells us:

- Number of entries.
- Column names.
- Non-null values.
- Data types.
- Memory usage.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   Row ID         9994 non-null   int64  
 1   Order ID       9994 non-null   object 
 2   Order Date     9994 non-null   object 
 3   Ship Date      9994 non-null   object 
 4   Ship Mode      9994 non-null   object 
 5   Customer ID    9994 non-null   object 
 6   Customer Name  9994 non-null   object 
 7   Segment        9994 non-null   object 
 8   Country        9994 non-null   object 
 9   City           9994 non-null   object 
 10  State          9994 non-null   object 
 11  Postal Code    9994 non-null   int64  
 12  Region         9994 non-null   object 
 13  Product ID     9994 non-null   object 
 14  Category       9994 non-null   object 
 15  Sub-Category   9994 non-null   object 
 16  Product Name   9994 non-null   object 
 17  Sales          9994 non-null   float64
 18  Quantity       9994 non-null   int64  
 19  Discount       9994 non-null   float64
 20  Profit         9994 non-null   float64
dtypes: float64(3), int64(3), object(15)
memory usage: 1.6+ MB
```

Then we check for any missing value I used df.isnull().sum() here isnull() identifies missing values and sum() will count how many

	0
Row ID	0
Order ID	0
Order Date	0
Ship Date	0
Ship Mode	0
Customer ID	0
Customer Name	0
Segment	0
Country	0
City	0
State	0
Postal Code	0
Region	0
Product ID	0
Category	0
Sub-Category	0
Product Name	0
Sales	0
Quantity	0
Discount	0
Profit	0

dtype: int64

missing values in our dataset

Then check for any duplicated records using df.duplicated().sum()

Check Duplicated Records(rows):

```
df.duplicated().sum()
```

```
np.int64(0)
```

Then to overview the whole dataset I used def.describe()

Overview of Dataset:

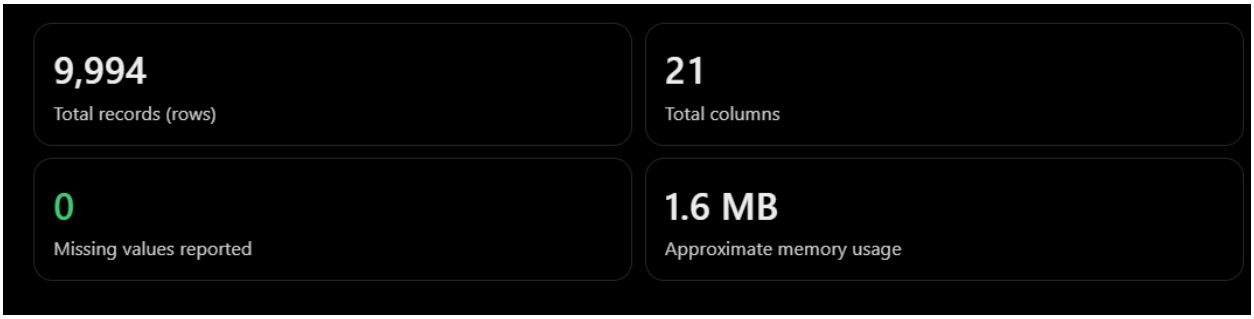
This generates basic statistical information for numerical columns.

It includes: Statistic Meaning count Number of non-missing values mean Average std Standard deviation min Minimum value 25% First quartile 50% Median 75% Third quartile max Maximum value

```
df.describe()
```

	Row ID	Postal Code	Sales	Quantity	Discount	Profit
count	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000
mean	4997.500000	55190.379428	229.858001	3.789574	0.156203	28.656896
std	2885.163629	32063.693350	623.245101	2.225110	0.206452	234.260108
min	1.000000	1040.000000	0.444000	1.000000	0.000000	-6599.978000
25%	2499.250000	23223.000000	17.280000	2.000000	0.000000	1.728750
50%	4997.500000	56430.500000	54.490000	3.000000	0.200000	8.666500
75%	7495.750000	90008.000000	209.940000	5.000000	0.200000	29.364000
max	9994.000000	99301.000000	22638.480000	14.000000	0.800000	8399.976000

The final basic understanding of dataset is



- We have 9,994 sales records.
- Each record contains 21 attributes.
- All columns have 9,994 non-null values, meaning no missing values were reported by info().
- Our dataset contains both numerical and categorical information.

Metric	Result
Average sales per row	229.86
Average quantity	3.79
Average discount	15.62%
Average profit per row	28.66
Highest sales value	22,638.48
Highest profit	8,399.976
Lowest profit	-6,599.978

An interesting observation: The minimum profit is negative.

This means some sales records resulted in losses rather than profits.

Now we will investigate that which category or region has more losses.

The average sales value (229.86) is much higher than the median sales value (54.49). This suggests that some unusually large sales records are pulling the average upward.

Its just initial observations, not final findings.

Now we start data filtering:

Filtering means selecting only those records that satisfy a particular condition.

Then the first filtering is to find the high sales rcords upto 500

Find High Sales Values

Sales greater than 500 Rs

```
high_sales = df[df['Sales'] > 500]

high_sales.head()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount	Profit
1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-CH-10000454	Furniture	Chairs	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400	3	0.00	219.5820
3	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311	South	FUR-TA-10000577	Furniture	Tables	Bretford CR4500 Series Slim Rectangular Table	957.5775	5	0.45	-383.0310
7	8	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	90032	West	TEC-PH-10002275	Technology	Phones	Mitel 5320 IP Phone VoIP phone	907.1520	6	0.20	90.7152
10	11	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	90032	West	FUR-TA-10001539	Furniture	Tables	Chromcraft Rectangular Conference Tables	1706.1840	9	0.20	85.3092
11	12	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	...	90032	West	TEC-PH-10002033	Technology	Phones	Konftel 250 Conference phone - Charcoal black	911.4240	4	0.20	68.3568

5 rows × 21 columns

The total rows are 1162 this much that had high sales >500 I find it using len(high_sales)

This is something we find from this

Your result:

- Total original records: 9,994
- Records with sales greater than 500: 1,162
- Displayed records: 5 (because we used `head()`)

Let's calculate the percentage of high-sales records:

$$\frac{1162}{9994} \times 100 \approx 11.63\%$$

High-sales records

11.63%

1,162 out of 9,994 records have sales above 500.

An interesting observation

Look at your filtered output:

- A Furniture product generated sales of 957.5775 but had a loss of -383.031.
- Another Furniture product generated sales of 731.94 and profit of 219.582.

This demonstrates something interesting:

High sales do not necessarily mean high profit.

Then we filter the sales that are in loss or <0 negative

```
loss_records = df[df['Profit'] < 0]

loss_records.head()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount	Profit
3	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311	South	FUR-TA-10000577	Furniture	Tables	Bretford CR4500 Series Slim Rectangular Table	957.5775	5	0.45	-383.0310
14	15	US-2015-118983	11/22/2015	11/26/2015	Standard Class	HP-14815	Harold Pawlan	Home Office	United States	Fort Worth	...	76106	Central	OFF-AP-10002311	Office Supplies	Appliances	Holmes Replacement Filter for HEPA Air Cleaner...	68.8100	5	0.80	-123.8580
15	16	US-2015-118983	11/22/2015	11/26/2015	Standard Class	HP-14815	Harold Pawlan	Home Office	United States	Fort Worth	...	76106	Central	OFF-BI-10000756	Office Supplies	Binders	Storex DuraTech Recycled Plastic Frosted Binders	2.5440	3	0.80	-3.8160
23	24	US-2017-156909	7/16/2017	7/18/2017	Second Class	SF-20065	Sandra Flanagan	Consumer	United States	Philadelphia	...	19140	East	FUR-CH-10002774	Furniture	Chairs	Global Deluxe Stacking Chair, Gray	71.3720	2	0.30	-1.0196
27	28	US-2015-150630	9/17/2015	9/21/2015	Standard Class	TB-21520	Tracy Blumstein	Consumer	United States	Philadelphia	...	19140	East	FUR-BO-10004834	Furniture	Bookcases	Riverside Palais Royal Lawyers Bookcase, Royal...	3083.4300	7	0.50	-1665.0522

5 rows × 21 columns

Your results:

Metric	Value
Total sales records	9,994
Loss-making records	1,871
Loss-making percentage	18.72%
Non-loss-making records	8,123
Non-loss-making percentage	81.28%

Distribution of sales records

Loss-making **18.72%**

Zero or positive profit **81.28%**

What have we learned?

Approximately 19 out of every 100 sales records resulted in a loss.

Notice that this is a percentage of **records**, not a percentage of total revenue or total business orders.

Then we started data sorting:

First we sort sales with high sales values

```
top_sales = df.sort_values(by='Sales', ascending=False)

top_sales.head()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount	Profit
2697	2698	CA-2014-145317	3/18/2014	3/23/2014	Standard Class	SM-20320	Sean Miller	Home Office	United States	Jacksonville	...	32216	South	TEC-MA-10002412	Technology	Machines	Cisco TelePresence System EX90 Videoconferenci...	22638.480	6	0.5	-1811.0784
6826	6827	CA-2016-118689	10/2/2016	10/9/2016	Standard Class	TC-20980	Tamara Chand	Corporate	United States	Lafayette	...	47905	Central	TEC-CO-10004722	Technology	Copiers	Canon imageCLASS 2200 Advanced Copier	17499.950	5	0.0	8399.9760
8153	8154	CA-2017-140151	3/23/2017	3/25/2017	First Class	RB-19360	Raymond Buch	Consumer	United States	Seattle	...	98115	West	TEC-CO-10004722	Technology	Copiers	Canon imageCLASS 2200 Advanced Copier	13999.960	4	0.0	6719.9808
2623	2624	CA-2017-127180	10/22/2017	10/24/2017	First Class	TA-21385	Tom Ashbrook	Home Office	United States	New York City	...	10024	East	TEC-CO-10004722	Technology	Copiers	Canon imageCLASS 2200 Advanced Copier	11199.968	4	0.2	3919.9888
4190	4191	CA-2017-166709	11/17/2017	11/22/2017	Standard Class	HL-15040	Hunter Lopez	Consumer	United States	Newark	...	19711	East	TEC-CO-10004722	Technology	Copiers	Canon imageCLASS 2200 Advanced Copier	10499.970	3	0.0	5039.9856

5 rows × 21 columns

And what we observe from it that the highest sale record has 1811.0784 loss it proved our observation that high sales not always means high profits.

Then we sort by profit means the sales records that has highest profit and to lowest profit

Sort by Profit:

Sort records who has highest profits to lower profits.

```
top_profit = df.sort_values(by='Profit', ascending=False)

top_profit.head()
```

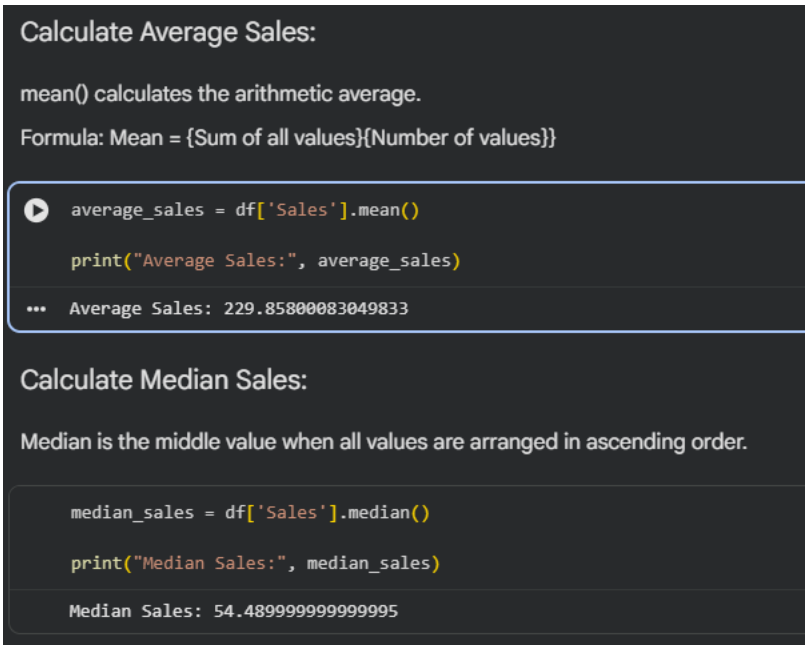
	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount	Profit
6826	6827	CA-2016-118689	10/2/2016	10/9/2016	Standard Class	TC-20980	Tamara Chand	Corporate	United States	Lafayette	...	47905	Central	TEC-CO-10004722	Technology	Copiers	Canon imageCLASS 2200 Advanced Copier	17499.95	5	0.0	8399.9760
8153	8154	CA-2017-140151	3/23/2017	3/25/2017	First Class	RB-19360	Raymond Buch	Consumer	United States	Seattle	...	98115	West	TEC-CO-10004722	Technology	Copiers	Canon imageCLASS 2200 Advanced Copier	13999.96	4	0.0	6719.9808
4190	4191	CA-2017-166709	11/17/2017	11/22/2017	Standard Class	HL-15040	Hunter Lopez	Consumer	United States	Newark	...	19711	East	TEC-CO-10004722	Technology	Copiers	Canon imageCLASS 2200 Advanced Copier	10499.97	3	0.0	5039.9856
9039	9040	CA-2016-117121	12/17/2016	12/21/2016	Standard Class	AB-10105	Adrian Barton	Consumer	United States	Detroit	...	48205	Central	OFF-BI-10000545	Office Supplies	Binders	GBC Ibimaster 500 Manual ProClick Binding System	9892.74	13	0.0	4946.3700
4098	4099	CA-2014-116904	9/23/2014	9/28/2014	Standard Class	SC-20095	Sanjit Chand	Consumer	United States	Minneapolis	...	55407	Central	OFF-BI-10001120	Office Supplies	Binders	Ibico EPK-21 Electric Binding System	9449.95	5	0.0	4630.4755

5 rows × 21 columns

We found that the top profit making one is the second high sales instead of the one that has highest sales records .

Now we move to Statistical Analysis:

Here we first find the mean of sales which is 220.86 and then median that is 54.49



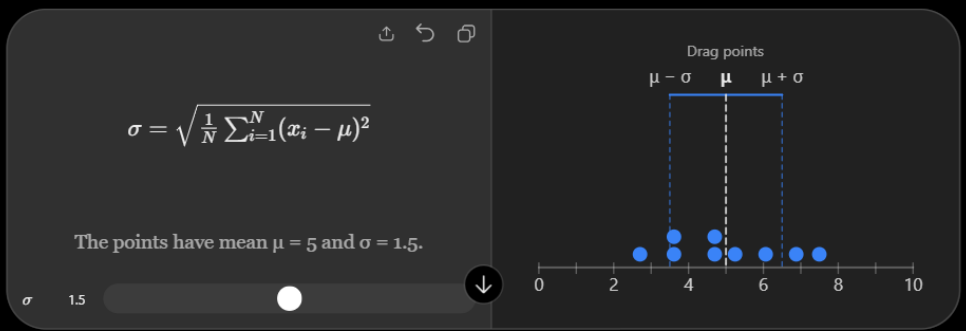
The difference suggests that some large sales transactions are increasing the average.

Then I calculated the Standard Devision of sales and profits and the results says the following things:

Metric	Result	Interpretation
Sales Standard Deviation	623.25	Sales values have a wide spread around the average.
Profit Standard Deviation	234.26	Profit values also vary considerably.
Average Quantity	3.79	Each record represents approximately 3.79 units on average.

Roman Urdu explanation

- **Sales Standard Deviation:** Average sales 229.86 hain, lekin standard deviation 623.25 hai. Iska matlab sales values mein bohat variation hai. Kuch transactions ki sales bohat high hain, jabke bohat si transactions ki sales comparatively low hain.
- **Profit Standard Deviation:** Profit mein bhi kaafi variation hai. Kuch records mein high profit hai aur kuch mein significant loss.
- **Average Quantity:** Har record mein average quantity approximately 3.79 units hai. Isay hum roughly 4 units keh sakte hain, lekin exact result 3.79 hi report karna hai.



Then we find profits by categories to find that which category has highest sales and profits

Category-wise Analysis:

```
category_analysis = df.groupby('Category')[['Sales', 'Profit']].sum()

category_analysis
```

Category	Sales	Profit
Furniture	741999.7953	18451.2728
Office Supplies	719047.0320	122490.8008
Technology	836154.0330	145454.9481

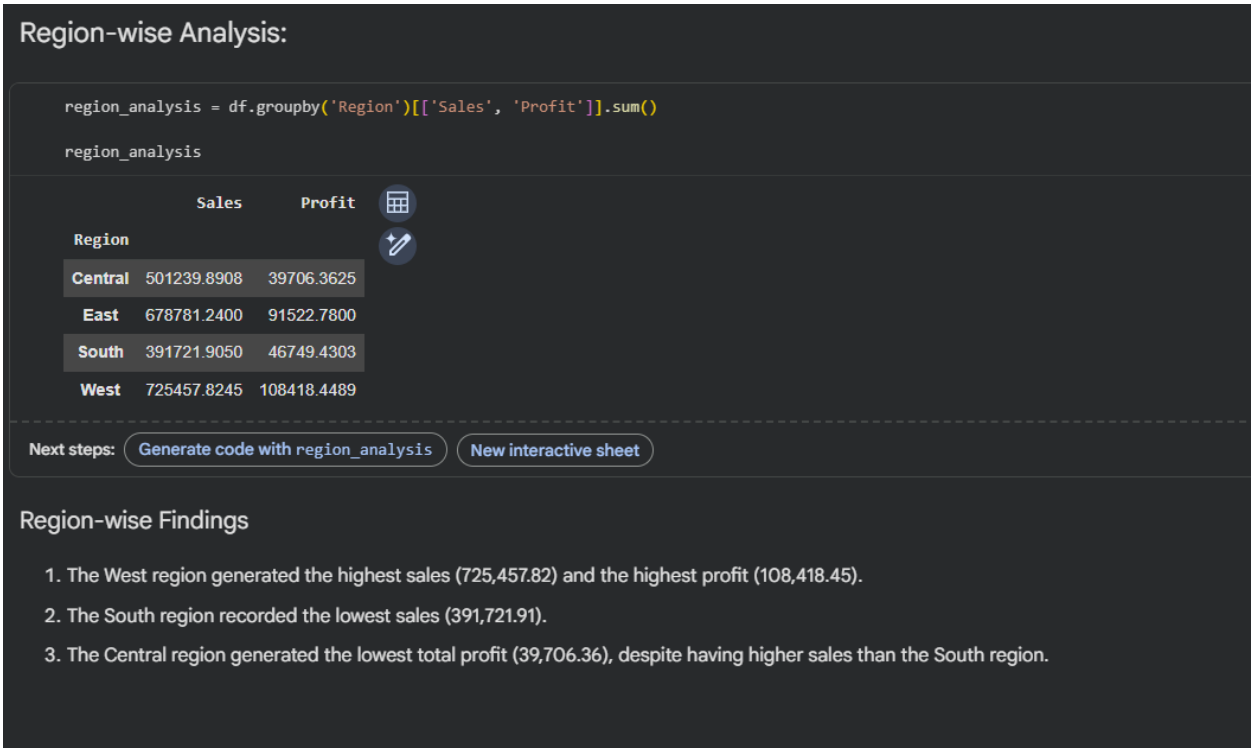
Next steps: Generate code with category_analysis | New interactive sheet

Category-wise Findings

1. Technology generated the highest total sales (836,154.03) and the highest total profit (145,454.95).
2. Furniture generated 741,999.80 in sales but only 18,451.27 in profit, showing a relatively small profit compared with its sales.
3. Office Supplies generated lower sales than Furniture but significantly higher profit (122,490.80 compared with 18,451.27).

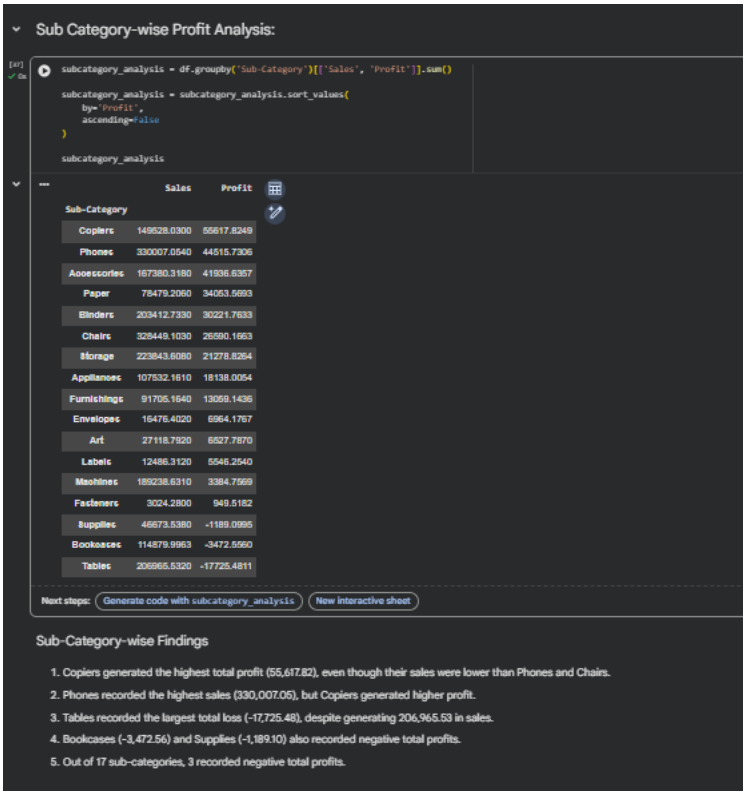
And we find that Technology has highest sales and profits while Furniture has high sales but has lowest profits while Office supplies has low sales comparatives to others but has sales higher than furniture.

Then we do the region based analysis :

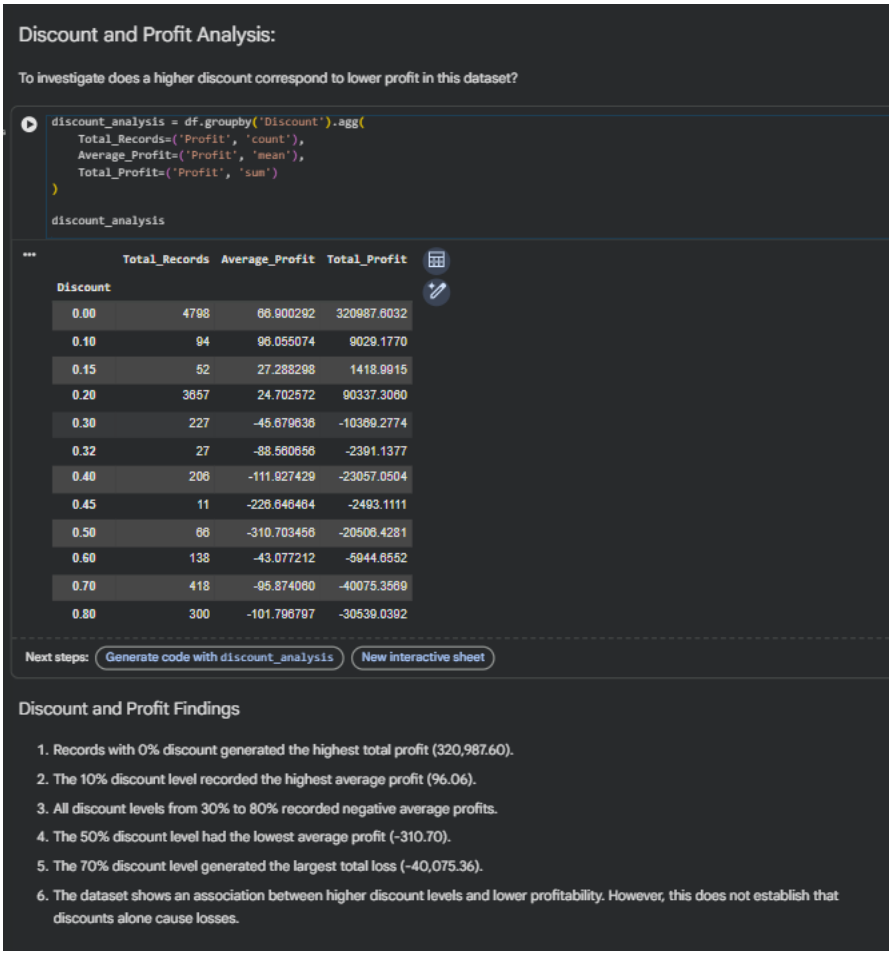


The results show that west has highest sales and profits while south has lowest sales and the central has high sales compare to south but has lower profit.

Then we do analysis sub categories wise



Then we do the analysis with discount and profit:



Discount and Profit Analysis:

To investigate does a higher discount correspond to lower profit in this dataset?

```
discount_analysis = df.groupby('Discount').agg(
    Total_Records=('Profit', 'count'),
    Average_Profit=('Profit', 'mean'),
    Total_Profit=('Profit', 'sum')
)

discount_analysis
```

Discount	Total_Records	Average_Profit	Total_Profit
0.00	4798	66.900292	320987.6032
0.10	94	96.055074	9029.1770
0.15	52	27.288298	1418.9915
0.20	3657	24.702572	90337.3060
0.30	227	-45.679636	-10369.2774
0.32	27	-88.560656	-2391.1377
0.40	206	-111.927429	-23057.0504
0.45	11	-226.646464	-2493.1111
0.50	66	-310.703456	-20506.4281
0.60	138	-43.077212	-5944.6552
0.70	418	-95.874060	-40075.3569
0.80	300	-101.796797	-30539.0392

Next steps: Generate code with discount_analysis | New interactive sheet

Discount and Profit Findings

1. Records with 0% discount generated the highest total profit (320,987.60).
2. The 10% discount level recorded the highest average profit (96.06).
3. All discount levels from 30% to 80% recorded negative average profits.
4. The 50% discount level had the lowest average profit (-310.70).
5. The 70% discount level generated the largest total loss (-40,075.36).
6. The dataset shows an association between higher discount levels and lower profitability. However, this does not establish that discounts alone cause losses.

Then we do Customer segment analysis:

Customer Segment Analysis

Now let's analyze the three customer segments:

- Consumer
- Corporate
- Home Office We want to know which customer segment contributes the most sales and profit.

```
segment_analysis = df.groupby('Segment').agg(
    Total_Sales=('Sales', 'sum'),
    Total_Profit=('Profit', 'sum'),
    Average_Sales=('Sales', 'mean'),
    Average_Profit=('Profit', 'mean')
)

segment_analysis
```

Segment	Total_Sales	Total_Profit	Average_Sales	Average_Profit
Consumer	1.161401e+06	134119.2092	223.783644	25.836873
Corporate	7.061464e+05	91979.1340	233.823300	30.456687
Home Office	4.296531e+05	60298.6785	240.972041	33.818664

Next steps: Generate code with segment_analysis | New interactive sheet

Customer Segment Findings

1. The Consumer segment generated the highest total sales (1,161,401.35) and total profit (134,119.21).
2. The Home Office segment recorded the highest average sales per record (240.97).
3. The Home Office segment also achieved the highest average profit per record (33.82).
4. Although Consumer leads in total profit, Home Office performs better in average profit per record.

```
yearly_analysis = df.groupby('Year').agg(
    Total_Sales=('Sales', 'sum'),
    Total_Profit=('Profit', 'sum'),
    Average_Sales=('Sales', 'mean'),
    Average_Profit=('Profit', 'mean')
)

yearly_analysis
```

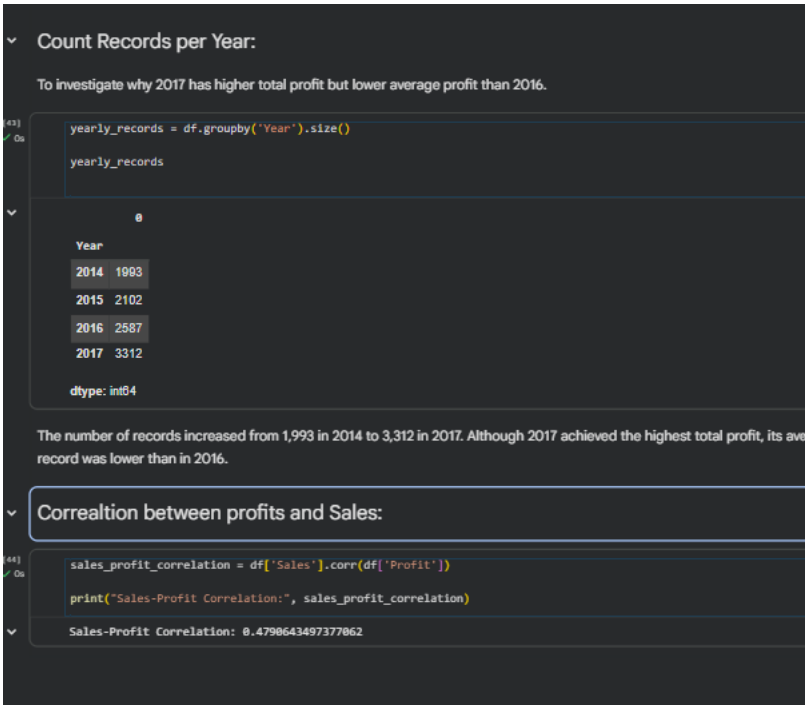
Year	Total_Sales	Total_Profit	Average_Sales	Average_Profit
2014	484247.4981	49543.9741	242.974159	24.858994
2015	470532.5090	61618.6037	223.849909	29.314274
2016	609205.5980	81795.1743	235.487282	31.617771
2017	733215.2552	93439.2696	221.381418	28.212340

Next steps: Generate code with yearly_analysis | New interactive sheet

Year-wise Findings

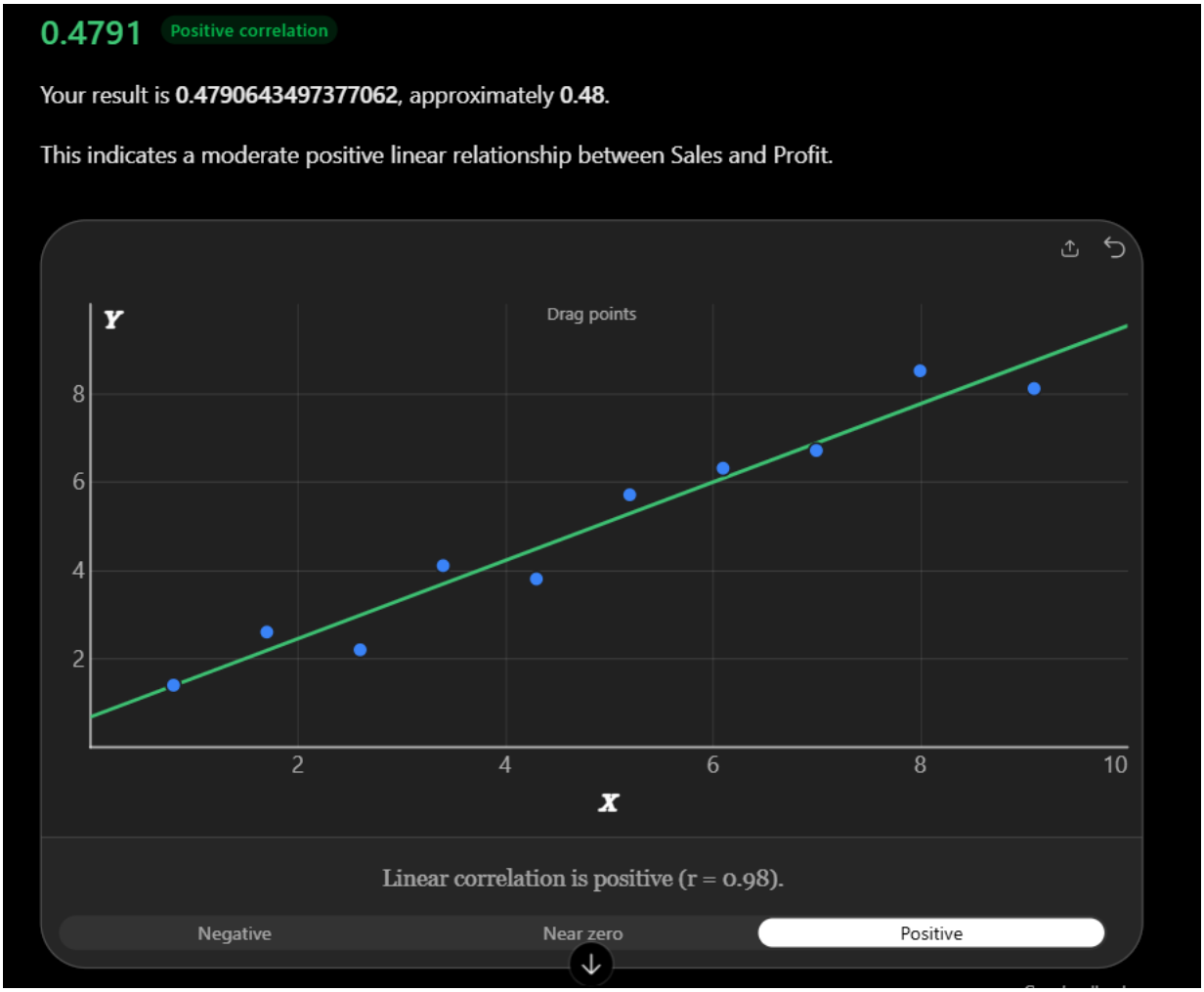
1. The year 2017 recorded the highest total sales (733,215.26) and total profit (93,439.27).
2. In 2015, total sales decreased compared with 2014, but total profit increased from 49,543.97 to 61,618.60.
3. The year 2016 achieved the highest average profit per record (31.62).
4. Average profit per record decreased from 31.62 in 2016 to 28.21 in 2017, despite an increase in total profit.

Then we did count records year wise and then find the correlation of profits and sales:



Correlation tells us that numerical varaibales has in which direction their relationship is:

- Positive correlation: aik variable increase ho to doosra generally increase hota hai.
- Negative correlation: aik increase ho to doosra generally decrease hota hai.
- Near-zero correlation: strong linear relationship nazar nahi aata.

- 
0.4791
Positive correlation
Your result is 0.4790643497377062, approximately 0.48.
This indicates a moderate positive linear relationship between Sales and Profit.
Drag points
y
8
6
4
2
2
4
6
8
10
x
Linear correlation is positive (r = 0.98).
Negative
Near zero
Positive